## Ex. 3.9 (20 pts) — three papers from 'Memory Management'

I read papers 3-1, 3-6 and 3-4, chosen because between them they follow one idea through three eras: the behavioural model that told an OS what to keep resident, the kernel structure that made paging portable, and the hardware pressure now reshaping both. (The list truncates 3-6's title and mistypes 3-2's; the citations below are the published ones.)

### 1. The working set model (paper 3-1)

**Peter J. Denning**, "The Working Set Model for Program Behavior", *Communications of the ACM* **11**(5):323–333, May 1968. First presented at the ACM Symposium on Operating System Principles, Gatlinburg, TN, October 1967.

**Summary.** Resource allocation had no theory, Denning argues, because program behaviour had no model, and the model cannot come from outside the program: users cannot cost shared, data-dependent code, and compilers cannot see across separately compiled modules. The system must measure behaviour itself.

He defines the working set $W(t,\tau)$ as the pages a process referenced during the process-time interval $(t-\tau,t)$, with size $\omega(t,\tau)$. It is monotone increasing and concave in $\tau$, with $\omega(t,2\tau) \leq 2\omega(t,\tau)$ on average; $W(t,\tau)$ predicts $W(t+\alpha,\tau)$ for $\alpha$ small against $\tau$ but not for $\alpha \gg \tau$; and from the distribution $F_x$ of the interval between successive references to a page, pages re-enter the working set at rate $\lambda(\tau) = [1-F_x(\tau)]/\bar{x}$, an estimate of page traffic. Setting $\tau$ trades that traffic against wasted memory, and since a page evicted just before re-reference costs two traverse times to return, he recommends a $\tau$ comparable to the traverse time $T$, $\tau \approx 2T$ being the value that holds the resulting drop in page residency to 50%.

Only $\omega$ is needed in practice, and look-ahead is rejected because a working set changes radically whenever its process blocks for interaction. Each page-table entry carries $K$ use bits shifted once per sampling interval $\sigma = \tau/K$; their logical OR is 1 exactly for the pages of $W(t,K\sigma)$, and a *checker* process in the running list evicts the rest. Memory management then folds into scheduling: memory demand is $\min(\omega/M, 1)$, processor demand follows from a hyperexponential model of CPU time between interactions, and a *balance* policy admits jobs so total demand matches the equipment, memory balanced first, since over-committing it produces thrashing.

**Critique.** The quantitative core sits in tension with the qualitative one. The traffic estimates at equations (8)–(11) assume successive interreference intervals are independent and identically distributed, whereas the prediction property, which is what makes a working set worth keeping, asserts the opposite, that "references to the same page tend to cluster in time". Denning does call these quantities estimates, but they rest on an assumption the rest of the paper denies, and they are the model's only quantitative ground.

That matters because the paper offers nothing else. Its conclusion ranks the four policies outright, "Random brings on the highest page traffic, working set the lowest", and asserts that the working set's extra cost "is more than offset by its accuracy". No experiment here tests the ranking. Denning does lean on measurement elsewhere, invoking Fine et al. and Varian and Coffman in Section 2, but the ranking itself rests on one sentence of the conclusion citing Oppenheimer and Weizer's simulation of the RCA Spectra 70/46: a citation, not a demonstration.

He also notes that with $l$ processes on $N$ processors, page tables are sampled every $l\sigma/N$ seconds rather than every $\sigma$, and his appendix scales the hardware timer to match; the software scheme gets

no such correction, so its effective $\tau$ stretches with load, in the overload regime the model exists to handle.

## 2. Machine-independent virtual memory in Mach (paper 3-6)

**R. Rashid, A. Tevanian, M. Young, D. Golub, R. Baron, D. Black, W. Bolosky and J. Chew**, "Machine-Independent Virtual Memory Management for Paged Uniprocessor and Multiprocessor Architectures", *Proc. 2nd Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, Palo Alto, CA, October 1987, pp. 31–39.

**Summary.** Instruction sets had stopped obstructing portability; memory-management units had not. The authors observe that Berkeley derivatives on non-VAX hardware internally *simulate* the VAX mapping architecture, promoting one vendor's MMU into a portability specification. Mach's answer is to hold every fact about virtual memory in machine-independent structures and confine machine dependence to one module, `pmap.c`. Four structures carry the design: a resident page table treating physical memory as a cache over memory objects; an *address map* of entries mapping virtual ranges onto byte ranges of a memory object, with their protection and inheritance; the *memory object*, a file-like repository of backing store; and the *pmap*, the hardware map.

The decisive move is that the pmap holds only a *cache* of translations. Mappings may be discarded at any moment and rebuilt lazily at fault time from the machine-independent structures; only kernel mappings must be complete. That is what keeps the machine-dependent module to roughly 6K bytes on a VAX, "about the size of a device driver", while the portable layer still supplies copy-on-write through chained shadow objects, mapped files, and external pagers running as ordinary user tasks.

The evidence is a port record: one source set runs on the VAX family, the IBM RT PC, the Sun 3, the Encore MultiMax and the Sequent Balance, with the RT PC `pmap` written in about three weeks by a programmer new to both operating systems and C. Section 5 then turns the portable base around on the hardware and finds every MMU wanting: the RT PC permits one mapping per physical page, and no multiprocessor keeps TLBs consistent.

**Critique.** The measurements do not test the claim they are enlisted for: what is asserted is that the machine-independent/dependent split costs nothing, what is measured is Mach against 4.3bsd, SunOS and ACIS. The wins are real but orthogonal to portability: a 256K `fork` takes 41 ms against 145 ms because of copy-on-write, not because of `pmap`. The experiment that would settle it, Mach against a hand-written machine-specific Mach, is absent. One confound they do handle: beside the 4:1 compile margin under 4.3bsd's generic buffers they report a 400-buffer run at 23 against 28 seconds.

The same gap reopens in Section 5. The RT PC's single-mapping restriction forces shared pages to be remapped on each alternation between tasks, and the evidence that these faults stay "rare enough in normal application programs" is again an end-to-end win, over IBM's ACIS 4.2a, which no more separates that cost than the fork figures do. The fault rate itself is never reported.

Section 3.5 names shadow chains as the design's main source of complexity. Mach does collapse them, garbage-collecting a shadow once the intermediate is recognised as redundant, and the authors are candid that the collector is incomplete: chains "sometimes occur during periods of heavy paging and cannot always be detected on the basis of in memory data structures alone". The discussion turns from there to that collector's locking rules, leaving the admitted residue with no bound on chain length and no cost, while every benchmark runs with memory to spare.

## 3. Transparent superpage support (paper 3-4)

**Juan Navarro, Sitaram Iyer, Peter Druschel and Alan Cox**, "Practical, Transparent Operating System Support for Superpages", *Proc. 5th USENIX Symposium on Operating Systems Design and Implementation (OSDI '02)*, Boston, MA, December 2002, pp. 89–104.

**Summary.** TLB coverage as a fraction of main memory fell by about a factor of 100 per decade between 1986 and 2001, and TLB misses that cost 4–5% of runtime in the 1980s now cost 30–60%. Superpages restore coverage, but a single large page size inflates footprints and paging granularity, so the operating system must mix sizes, and mixing sizes fragments the very contiguity large sizes need. The paper's answer is to treat contiguity as a contended resource.

On a fault the system reserves a contiguous aligned extent from a buddy allocator, as large as fits inside the faulting memory object without overlapping an existing reservation. Promotion is incremental: an aligned subregion is promoted once fully populated, then re-promoted at each larger size, and demoted again when the page daemon evicts one of its base pages. Under pressure, reservations are preempted and the page daemon wakes on low contiguity as well as low memory, preferentially reclaiming inactive pages that restore contiguity. Because the hardware provides one reference bit and one dirty bit per superpage, the system also demotes speculatively to find unused base pages, and demotes a clean superpage on first write, re-promoting only when every base page is dirty.

In 3500 lines on FreeBSD/Alpha, 18 of 35 benchmarks gain over 5% and 10 over 25%, and multiple sizes beat any single size (*mcf* 1.68 against 1.31 at best); after a web workload fragments memory, the contiguity-aware daemon lets FFTW reach a 55% speedup in two runs, where a cache-only scheme obtains no superpage at all.

**Critique.** The results are tied to one machine, in both directions. The Alpha's four page sizes are what the incremental design exploits, and Table 2 shows what survives with only the largest: *C4* falls from 1.36 to 0.98, *SP* from 1.19 to 0.98, *mcf* from 1.68 to 1.22. Section 2.2 names the i386 family as offering exactly two sizes and never returns to what that implies for the era's dominant platform. Conversely the overheads are inflated by Alpha PTE replication, charged with 7.2 of the adversary's 8.9 percentage points.

The abstract's "these benefits are sustained even under stressful workload scenarios" is likewise broader than the evidence: the long-term case replays one web access log and runs one benchmark four times, and the concurrent experiment substitutes a dummy allocator and disables promotion altogether, measuring contiguity restored rather than speedup sustained.

Demoting on write is a frank retreat and the authors say so, that it "may deny the benefits of superpages to processes that do not write to all of the base pages". The weaker move is the alternative they discard: hashing base pages to find which were really modified was rejected on preliminary microbenchmarks costing up to 15%, and 60% on a saturated sequential read, yet the same paragraph lists two optimisations, hashing only partially dirty superpages and moving the work into the idle loop, that attack exactly that cost and are left to future work. An option dismissed on its least optimised form has not been evaluated.